

✓ STEP 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report
```

✓ STEP 2: Load Dataset

```
df = pd.read_excel("/content/Dataset_Final_Project from PPDA.xlsx")
```

```
# View first rows
print(df.head())
```

[Show hidden output](#)

```
# Check structure
print(df.info())
```

[Show hidden output](#)

✓ STEP 3: Data Description

```
# Summary statistics
print(df.describe())
```

[Show hidden output](#)

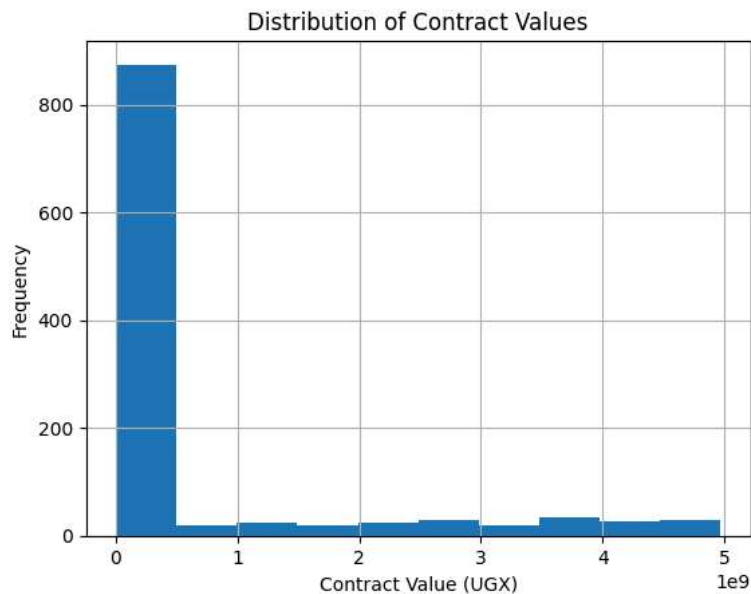
```
# Check unique values
print(df['Status'].value_counts())
print(df['Procurement_Method'].value_counts())
```

[Show hidden output](#)

✓ STEP 4: Data Visualization

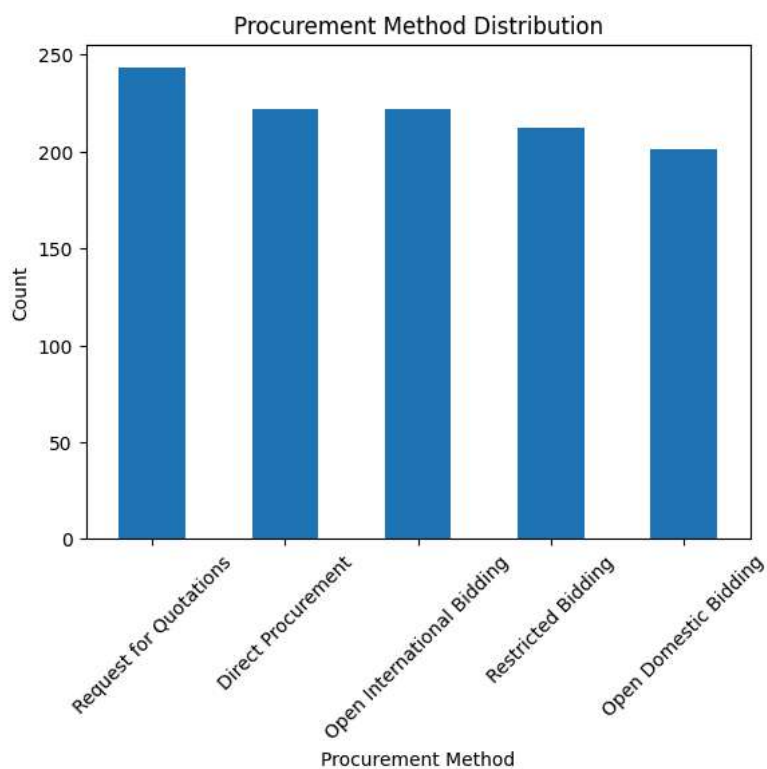
4.1 Univariate – Contract Value Distribution

```
plt.figure()
df['Contract_Value_UGX'].hist()
plt.xlabel("Contract Value (UGX)")
plt.ylabel("Frequency")
plt.title("Distribution of Contract Values")
plt.show()
```



4.2 Univariate – Procurement Method

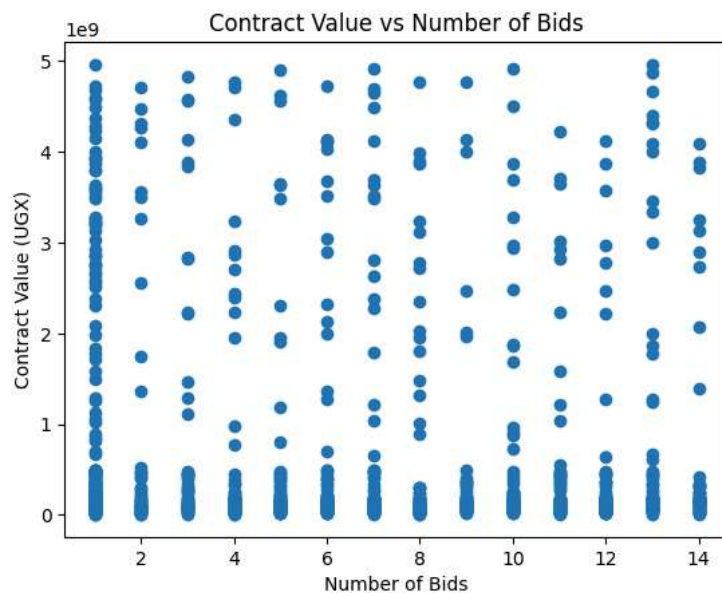
```
plt.figure()
df['Procurement_Method'].value_counts().plot(kind='bar')
plt.xlabel("Procurement Method")
plt.ylabel("Count")
plt.title("Procurement Method Distribution")
plt.xticks(rotation=45)
plt.show()
```



4.3 Multivariate – Value vs Bids

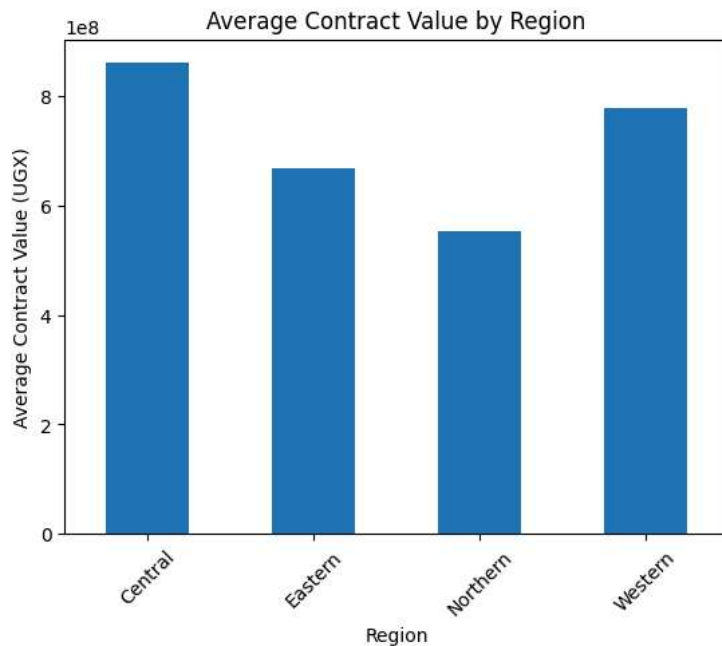
```
plt.figure()
plt.scatter(df['Number_of_Bids'], df['Contract_Value_UGX'])
plt.xlabel("Number of Bids")
plt.ylabel("Contract Value (UGX)")
```

```
plt.title("Contract Value vs Number of Bids")
plt.show()
```



4.4 Multivariate – Region vs Value

```
plt.figure()
df.groupby('Region')['Contract_Value_UGX'].mean().plot(kind='bar')
plt.xlabel("Region")
plt.ylabel("Average Contract Value (UGX)")
plt.title("Average Contract Value by Region")
plt.xticks(rotation=45)
plt.show()
```



✓ STEP 5: Data Cleaning & Preparation

```
df_ml = df.copy()

# Encode categorical variables
label_encoders = {}

categorical_cols = [
```

```

        'Procuring_Entity',
        'Category',
        'Item_Description',
        'Procurement_Method',
        'Region'
    ]

    for col in categorical_cols:
        le = LabelEncoder()
        df_ml[col] = le.fit_transform(df_ml[col])
        label_encoders[col] = le

    # Encode target variable
    le_status = LabelEncoder()
    df_ml['Status'] = le_status.fit_transform(df_ml['Status'])

```

▼ STEP 6: Feature Selection

```

X = df_ml.drop(['Contract_ID', 'Status'], axis=1)
y = df_ml['Status']

```

▼ STEP 7: Feature Scaling

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

▼ STEP 8: Train-Test Split

```

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

```

▼ STEP 9: Classical ML Model (Random Forest)

```

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

rf_preds = rf.predict(X_test)

print("Random Forest Results:")
print(classification_report(y_test, rf_preds))

```

```

Random Forest Results:
              precision    recall  f1-score   support

     0       0.71      0.99      0.82      154
     1       0.60      0.06      0.10       54
     2       0.00      0.00      0.00       12

 accuracy          0.70      0.70      0.70      220
 macro avg       0.44      0.35      0.31      220
 weighted avg     0.64      0.70      0.60      220

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

▼ STEP 10: Neural Network Model

```

mlp = MLPClassifier(max_iter=500)

mlp.fit(X_train, y_train)

mlp_preds = mlp.predict(X_test)

print("Neural Network Results:")
print(classification_report(y_test, mlp_preds))

```

Neural Network Results:

	precision	recall	f1-score	support
0	0.71	0.95	0.81	154
1	0.31	0.07	0.12	54
2	0.00	0.00	0.00	12
accuracy			0.68	220
macro avg	0.34	0.34	0.31	220
weighted avg	0.57	0.68	0.60	220

```

/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Opt
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

▼ STEP 11: Model Evaluation [Confusion Matrix]

```

from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test)
plt.title("Random Forest Confusion Matrix")
plt.show()

```

